

Lecture 8

Control Unit

“A SIMPLE COMPUTER ARCHITECTURE”

Introduction

- In the last lecture, we introduced datapaths for processing data under the supervision of a Control Unit.
- In this lecture, our focus will be on the design of the control unit.
- Digital systems can be classified as programmable and nonprogrammable systems depending on the type of control unit.
- Nonprogrammable systems have inputs, but do not have any mechanism for executing a programs (e.g., fixed Multiplier system).
- Programmable systems are capable of executing programs.

C.U in a Programmable System

- The control unit generates :
 - Sequence in which the instructions are performed.
 - Control word that activate the various instructions
- In a programmable system, a portion of the input to the processor consists of a sequence of *instructions*.
- Each instruction specifies:
 - An operation the system is to perform,
 - Which operands to use for the operation,
 - Where to place the results of the operation,
 - And/or, in some cases, which instruction to execute next.

Cont.,

- To execute the instructions in sequence, it is necessary to provide the address in memory of the instruction to be executed.
- In a computer, this address comes from a register called the *program counter (PC)*.
- So, in the case of a programmable system, the control unit contains a *PC* and associated decision logic, as well as the necessary logic to interpret the instruction in order to execute it.

C.U In a Nonprogrammable System

- In contrast to the preceding, note that for a nonprogrammable system, the control unit is not responsible for obtaining instructions from memory, nor is it responsible for sequencing the execution of those instructions.
- There is no *PC* or similar register in such a system.
- Instead, the control unit determines the operations to be performed and the sequence of those operations, based on only its inputs and the status bits.

C.U Design Approaches

- There are two different approaches to design a control unit:

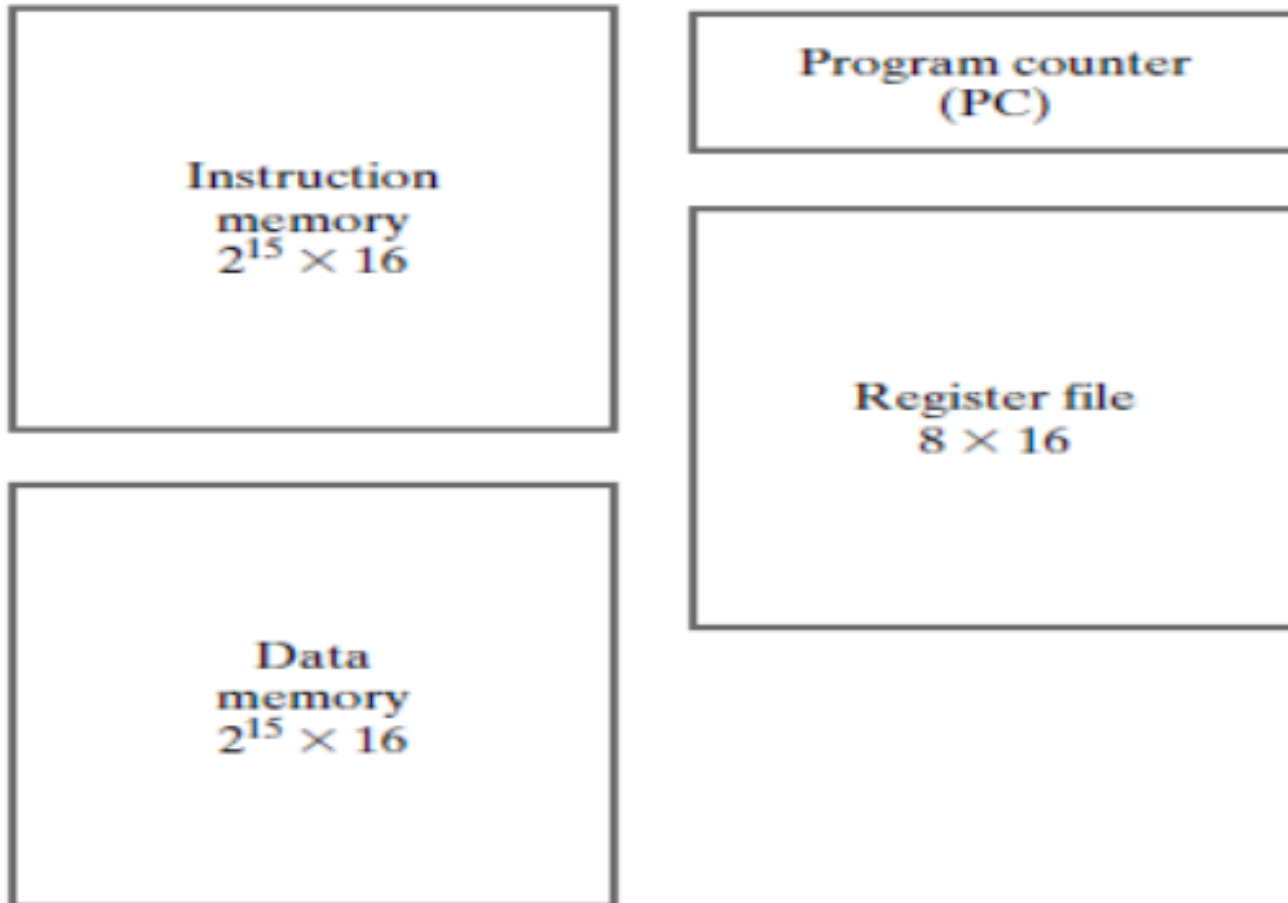
- ☐ A Hardwired Control Unit

- ☐ Microprogrammed Control Unit

Instruction Set Architecture

- The control unit reads an instruction from memory , decodes, and executes the instruction by issuing a sequence of one or more microoperations.
- Simple instruction set architectures have three major components:
 - the storage resources,
 - the instruction formats,
 - and the instruction specifications.

Storage Resources for a simple computer

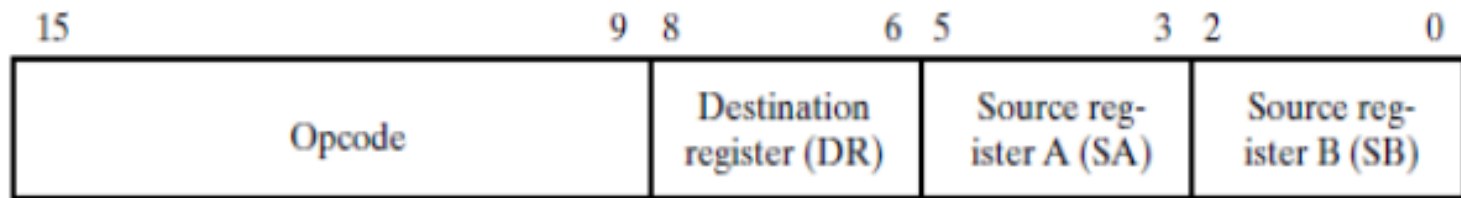


Instruction Formats

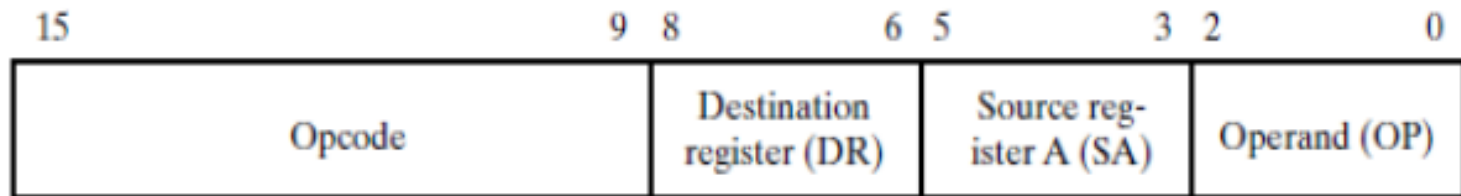
- The opcode of an instruction specifies the operation to be performed.
- The operation must be performed using data stored in computer registers or in memory.
- An instruction, therefore, must specify not only the operation, but also the registers or memory words in which the operands are to be found and the result is to be placed.

Instruction formats for the simple computer

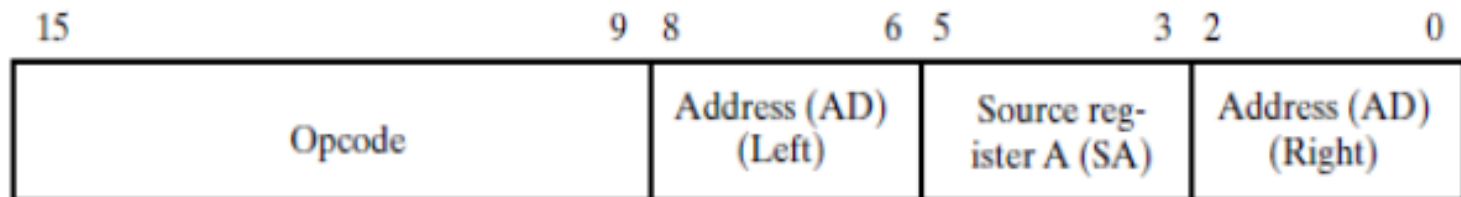
- Consider that we have three instruction formats:



(a) Register



(b) Immediate



(c) Jump and branch

Register Addressing Mode

- Suppose that the computer has a register file consisting of eight registers, $R0$ through $R7$.
- The instruction format in Figure (a) consists of an opcode that specifies the use of three or fewer registers, as needed.
- For example, for a subtraction operation, suppose that the three bits in SA are 010, specifying $R2$, the three bits in SB are 011, specifying $R3$, and the three bits in DR are 001, specifying $R1$.
- Then the contents of $R3$ will be subtracted from the contents of $R2$, and the result will be placed in $R1$.

Immediate Addressing Mode

- The instruction format in Figure (b) has an opcode, two register fields, and an operand.
- The operand is a constant called an *immediate operand*, since it is immediately available in the instruction.
- For example, for an add immediate operation with SA specified as *R7*, DR specified as *R2*, and operand OP equal to 011, the value 3 is added to the contents of *R7*, and the result of the addition is placed in *R2*.

Jump & Branch Mode

- The instruction format in Figure (c), in contrast to the other two formats, does not change any register file or memory contents.
- Instead, it affects the order in which the instructions are fetched from memory.
- The example format given in Figure (c) for jump and branch instructions has an operation code, one register field SA, and a split address field AD.
- If a branch is to occur, the new address is formed by adding the current PC contents and the contents of the 6-bit address field.

Cont.,

- This addressing method is called PC relative and the 6-bit address field, referred to as an *address offset*, is treated as a signed 2s complement number.
- To preserve the 2s complement representation, *sign extension* is applied to the 6-bit address to form a 16-bit offset before the addition.
- If the leftmost bit of the address field AD after converting it to its 2's complement form is 1, then the 10 bits to its left are filled with 1s to give a negative 2s complement offset.
- If the leftmost bit of the address field after 2's complement is 0, means the then the 10 bits to its left are filled with 0s to give a positive 2s complement offset (2's complement of 2's complement).
- The offset is added to the contents of the *PC* to form the location from which the next instruction is to be fetched.

Example

- With the *PC* value equal to 55, suppose that a branch is to occur to location 35 if the contents of *R6* is equal to zero.
- The opcode would specify a branch-on-zero instruction (JZ), SA would be specified as *R6*, and AD would be the 6-bit, 2s complement representation of -20 .
- $+20D = 0000000000\ 010\ 100$
- 2's complement of $20D = 1111111111\ 101\ 100$
- If *R6* is zero, then *PC* contents becomes $55 + (-20) = 35$, and the next instruction will be fetched from address 35.
- Otherwise, if *R6* is nonzero, the *PC* will count up to 56, and the next instruction will be fetched from address 56.

Instruction Specifications

- Instruction specifications describe each of the distinct instructions that can be executed by the system.
- For each instruction, the opcode is given along with a shorthand name called a mnemonic, which can be used as a symbolic representation for the opcode.
- This symbolic representation is then converted to the binary representation of the instruction by a program called an assembler.
- The instruction specifications for the simple computer are given in Table 8.

TABLE 8
Instruction Specifications for the Simple Computer

Instruction	Opcode	Mne- monic	Format	Description	Status Bits
Move A	0000000	MOVA	RD, RA	$R[DR] \leftarrow R[SA]^*$	N, Z
Increment	0000001	INC	RD, RA	$R[DR] \leftarrow R[SA] + 1^*$	N, Z
Add	0000010	ADD	RD, RA, RB	$R[DR] \leftarrow R[SA] + R[SB]^*$	N, Z
Subtract	0000101	SUB	RD, RA, RB	$R[DR] \leftarrow R[SA] - R[SB]^*$	N, Z
Decrement	0000110	DEC	RD, RA	$R[DR] \leftarrow R[SA] - 1^*$	N, Z
AND	0001000	AND	RD, RA, RB	$R[DR] \leftarrow R[SA] \wedge R[SB]^*$	N, Z
OR	0001001	OR	RD, RA, RB	$R[DR] \leftarrow R[SA] \vee R[SB]^*$	N, Z
Exclusive OR	0001010	XOR	RD, RA, RB	$R[DR] \leftarrow R[SA] \oplus R[SB]^*$	N, Z
NOT	0001011	NOT	RD, RA	$R[DR] \leftarrow \overline{R[SA]}^*$	N, Z
Move B	0001100	MOVB	RD, RB	$R[DR] \leftarrow R[SB]^*$	
Shift Right	0001101	SHR	RD, RB	$R[DR] \leftarrow sr\ R[SB]^*$	
Shift Left	0001110	SHL	RD, RB	$R[DR] \leftarrow sl\ R[SB]^*$	
Load Immediate	1001100	LDI	RD, OP	$R[DR] \leftarrow zf\ OP^*$	
Add Immediate	1000010	ADI	RD, RA, OP	$R[DR] \leftarrow R[SA] + zf\ OP^*$	N, Z
Load	0010000	LD	RD, RA	$R[DR] \leftarrow M[SA]^*$	
Store	0100000	ST	RA, RB	$M[SA] \leftarrow R[SB]^*$	
Branch on Zero	1100000	BRZ	RA, AD	if ($R[SA] = 0$) $PC \leftarrow PC + se\ AD$, if ($R[SA] \neq 0$) $PC \leftarrow PC + 1$	N, Z
Branch on Negative	1100001	BRN	RA, AD	if ($R[SA] < 0$) $PC \leftarrow PC + se\ AD$, if ($R[SA] \geq 0$) $PC \leftarrow PC + 1$	N, Z
Jump	1110000	JMP	RA	$PC \leftarrow R[SA]$	

* For all of these instructions, $PC \leftarrow PC + 1$ is also executed to prepare for the next cycle.

Note

- If a branch is taken, then the new *PC* value is the sum of the previous *PC* value and the sign-extended address offset, Otherwise, the *PC* is incremented by 1.
- A jump occurs if bit 13 in the instruction equal to 1.
- For bit 13 equal to 0, a conditional branch occurs.
- The status bit that is the condition for the branch is selected by bit 9 of the instruction.
- For bit 9 equal to 1, *N* is selected and, for bit 9 equal to 0, *Z* is selected.

Branch on Zero 1100000 BRZ

Branch on
Negative 1100001 BRN

Jump 1110000 JMP

Example

- Suppose that we have a memory with 16 bits per word and that the opcode contains 7 bits.
- Instructions and data, in binary, are placed in memory, as shown in Table 9.
- This stored information represents the four instructions illustrating the distinct formats.
- In memory location 35 is a register format instruction to store the contents of *R5* in the memory location specified by *R4*.
- In location 55, the branch instruction previously described appears.
- The opcode for this instruction is 96, and source register *A* is specified as *R6*.

Cont.,

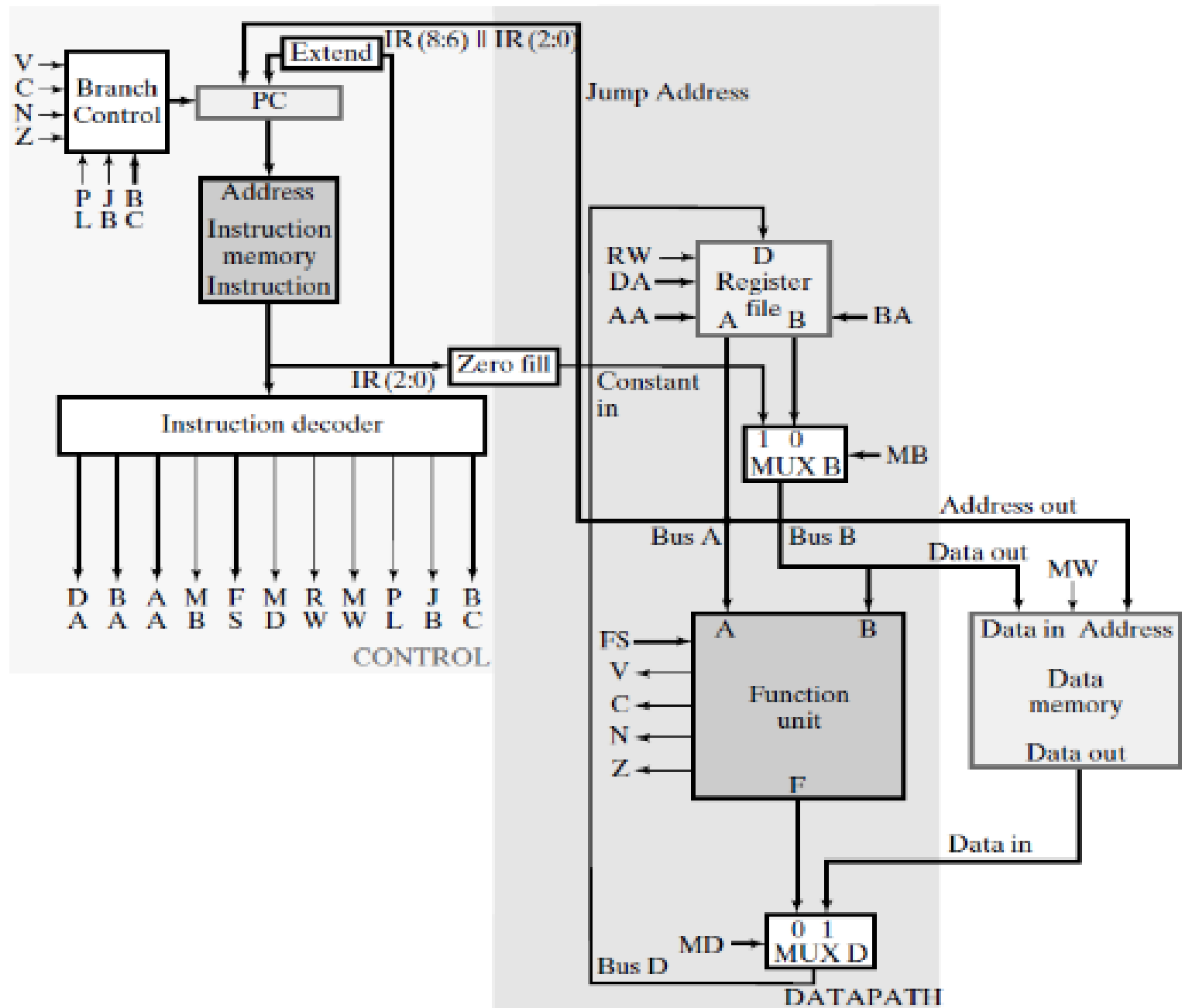
- Note that AD (Left) contains 101 and AD (Right) contains 100. Putting these two together and applying sign extension, we obtain 111111111101100, which represents -20 in 2s complement.
- Notice our assumption that the addition to the *PC* content occurs before the *PC* has been incremented, which would be the case in the simple computer.
- In real systems, however, the *PC* has sometimes been incremented to point to the next instruction in memory.
- In such a case, the value stored in AD needs to be adjusted accordingly to obtain the right branch address, in this case, -19.

TABLE 9
Memory Representation of Instructions and Data

Decimal Address	Memory Contents	Decimal Opcode	Other Fields	Operation
25		5 (Subtract)	DR:1, SA:2, SB:3	$R1 \leftarrow R2 - R3$
	0000101 001 010 011			
35		32 (Store)	SA:4, SB:5	$M[R4] \leftarrow R5$
	0100000 000 100 101			
45		66 (Add Immediate)	DR:2, SA:7, OP:3	$R2 \leftarrow R7 + 3$
	1000010 010 111 011			
55		96 (Branch on Zero)	AD: 44, SA:6	If $R6 = 0$, $PC \leftarrow PC - 20$
	1100000 101 110 100			
70		Data = 192. After execution of instruction in 35, Data = 80.		
	00000000011000000			

SINGLE-CYCLE HARDWIRED CONTROL

- The single-cycle computer is a computer that has a hardwired control unit that fetches and executes an instruction in a single clock cycle.
- The Datapath has $m = 3$ (register file is 8 registers) and $n = 16$.
- The data memory M is attached to the Address out, Data out, and Data in by connections to the datapath.
- It has a single control signal MW, which is 1 to write the memory, and 0 otherwise.



Cont.

- The Control unit appears on the left.
- the instruction memory, together with its address inputs and instruction outputs, is shown for convenience within the control unit.
- We do not write to the instruction memory during the execution of a program.
- The *PC* provides the instruction address to the instruction memory, and the instruction output from the instruction memory goes to the control logic, which in this case is the **instruction decoder**.
- If a jump occurs, the new *PC* value becomes the value on *Bus A*.

Cont.,

- The output from the instruction memory also goes to **Extend** and **Zero fill**, which provide the address offset to the *PC* and the constant input.
- The *PC* is updated in each clock cycle.
- The behavior of the *PC* is determined by the opcode, **N**, and **Z**, since *C* and *V* are not used in this control-unit design.
- there is no sequential logic in the control part other than the *PC*.

Instruction Decoder

- The instruction decoder is a combinational circuit that provides all of the control words for the datapath, based on the contents of the fields of the instruction.
- The following figure shows the Truth table for Instruction Decoder.

□ **TABLE 10**
Truth Table for Instruction Decoder Logic

Instruction Function Type	Instruction Bits				Control Word Bits						
	15	14	13	9	MB	MD	RW	MW	PL	JB	BC
Function-unit operations using registers	0	0	0	X	0	0	1	0	0	X	X
Memory read	0	0	1	X	0	1	1	0	0	X	X
Memory write	0	1	0	X	0	X	0	1	0	X	X
Function-unit operations using register and constant	1	0	0	X	1	0	1	0	0	X	X
Conditional branch on zero (<i>Z</i>)	1	1	0	0	X	X	0	0	1	0	0
Conditional branch on negative (<i>N</i>)	1	1	0	1	X	X	0	0	1	0	1
Unconditional jump	1	1	1	X	X	X	0	0	1	1	X

TABLE 8
Instruction Specifications for the Simple Computer

Instruction	Opcode	Mne- monic	Format	Description	Status Bits
Move A	0000000	MOVA	RD, RA	$R[DR] \leftarrow R[SA]^*$	N, Z
Increment	0000001	INC	RD, RA	$R[DR] \leftarrow R[SA] + 1^*$	N, Z
Add	0000010	ADD	RD, RA, RB	$R[DR] \leftarrow R[SA] + R[SB]^*$	N, Z
Subtract	0000101	SUB	RD, RA, RB	$R[DR] \leftarrow R[SA] - R[SB]^*$	N, Z
Decrement	0000110	DEC	RD, RA	$R[DR] \leftarrow R[SA] - 1^*$	N, Z
AND	0001000	AND	RD, RA, RB	$R[DR] \leftarrow R[SA] \wedge R[SB]^*$	N, Z
OR	0001001	OR	RD, RA, RB	$R[DR] \leftarrow R[SA] \vee R[SB]^*$	N, Z
Exclusive OR	0001010	XOR	RD, RA, RB	$R[DR] \leftarrow R[SA] \oplus R[SB]^*$	N, Z
NOT	0001011	NOT	RD, RA	$R[DR] \leftarrow \overline{R[SA]}^*$	N, Z
Move B	0001100	MOVB	RD, RB	$R[DR] \leftarrow R[SB]^*$	
Shift Right	0001101	SHR	RD, RB	$R[DR] \leftarrow sr\ R[SB]^*$	
Shift Left	0001110	SHL	RD, RB	$R[DR] \leftarrow sl\ R[SB]^*$	
Load Immediate	1001100	LDI	RD, OP	$R[DR] \leftarrow zf\ OP^*$	
Add Immediate	1000010	ADI	RD, RA, OP	$R[DR] \leftarrow R[SA] + zf\ OP^*$	N, Z
Load	0010000	LD	RD, RA	$R[DR] \leftarrow M[SA]^*$	
Store	0100000	ST	RA, RB	$M[SA] \leftarrow R[SB]^*$	
Branch on Zero	1100000	BRZ	RA, AD	if $(R[SA] = 0)$ $PC \leftarrow PC + se\ AD$, if $(R[SA] \neq 0)$ $PC \leftarrow PC + 1$	N, Z
Branch on Negative	1100001	BRN	RA, AD	if $(R[SA] < 0)$ $PC \leftarrow PC + se\ AD$, if $(R[SA] \geq 0)$ $PC \leftarrow PC + 1$	N, Z
Jump	1110000	JMP	RA	$PC \leftarrow R[SA]$	

* For all of these instructions, $PC \leftarrow PC + 1$ is also executed to prepare for the next cycle.

Where,

- the control-word fields DA, AA, and BA are equal to the instruction fields DR, SA, and SB, respectively.
- Since , this design uses only two status flags, N, and Z, therefore, BC is taken from bit #9 only.
- The bits 15:13 , are used for Instruction Function type.
- The remaining control-word fields include datapath and data memory control bits MB, MD, RW, and MW.

Cont.,

- Also, control field BC for selection of the branch condition status bits is taken directly from the last bit of Opcode.
- There are two added bits for the control of the *PC*: PL and JB.
- If there is to be a jump or branch, PL = 1, loading the *PC*.
- For PL = 0, the *PC* is incremented.
- With PL = 1, JB = 1 calls for a jump, and JB = 0 calls for a conditional branch.

Cont.,

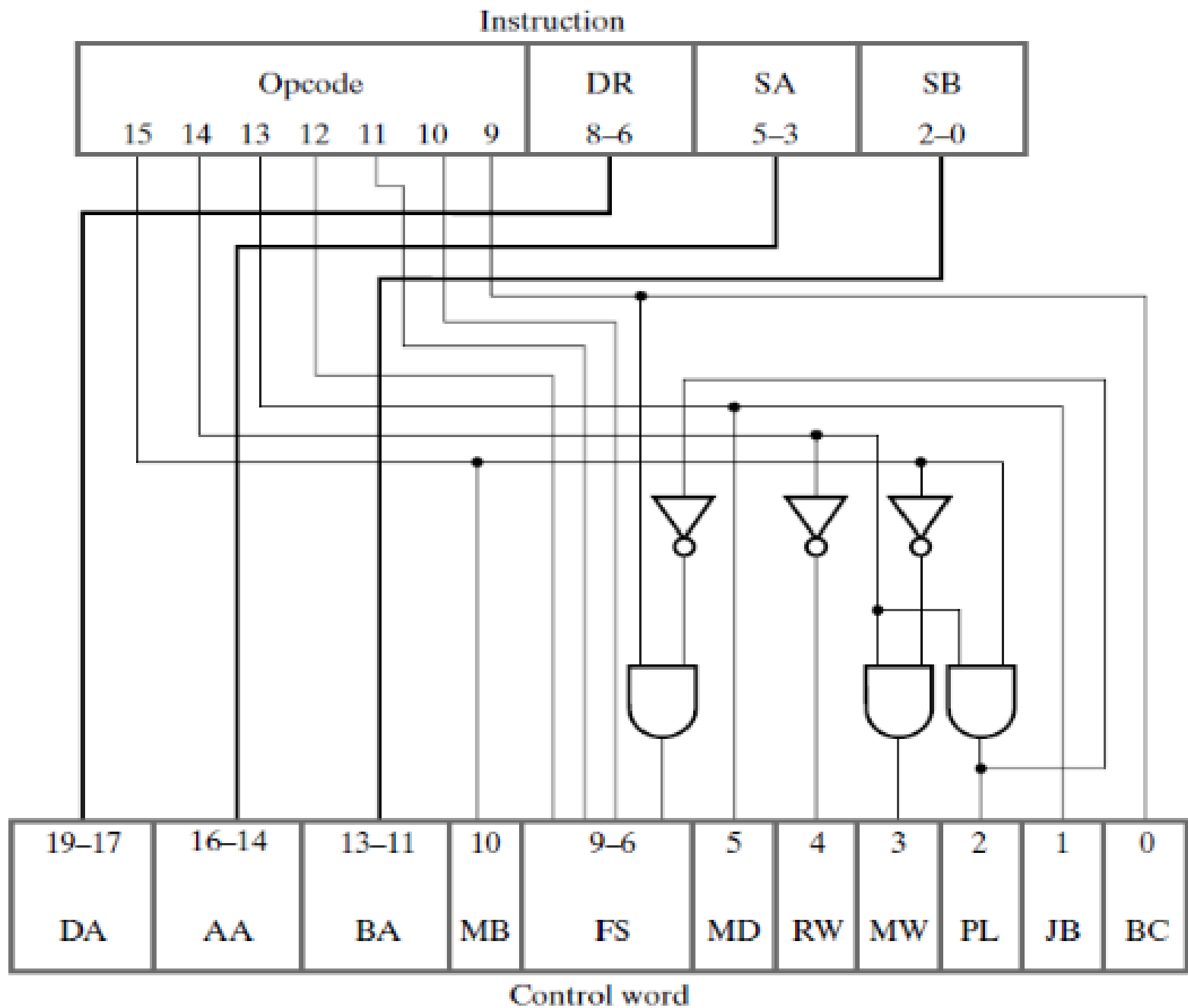
- From the Truth table, and by minimization:
 - $PL = \text{Bit \#15 AND Bit \#14}$
 - $JB = \text{Bit \# 13}$
 - $MB = \text{Bit \# 15}$
 - $MD = \text{Bit \# 13}$
 - $MW = \text{Bit \# 14 AND NOT(Bit \# 15)}$
 - $RW = \text{NOT (Bit \# 14)}$

□ **TABLE 10**
Truth Table for Instruction Decoder Logic

Instruction Function Type	Instruction Bits				Control Word Bits						
	15	14	13	9	MB	MD	RW	MW	PL	JB	BC
Function-unit operations using registers	0	0	0	X	0	0	1	0	0	X	X
Memory read	0	0	1	X	0	1	1	0	0	X	X
Memory write	0	1	0	X	0	X	0	1	0	X	X
Function-unit operations using register and constant	1	0	0	X	1	0	1	0	0	X	X
Conditional branch on zero (<i>Z</i>)	1	1	0	0	X	X	0	0	1	0	0
Conditional branch on negative (<i>N</i>)	1	1	0	1	X	X	0	0	1	0	1
Unconditional jump	1	1	1	X	X	X	0	0	1	1	X

Cont.,

- The remaining logic in the decoder deals with the FS field.
- For all but the conditional branch and unconditional jump instructions, bits 9 through 12 are fed directly through to form the FS field.
- During conditional branch operations, such as Branch on Zero, the value in source register *A* must be passed through the ALU so that the status bits *N* and *Z* can be evaluated. This requires FS = 0000.



Sample Instructions and Program

- Set of instructions for the single-cycle computer are listed in the following Table.
- The rightmost six columns of the table show critical control-signal values for each instruction.

□ TABLE 11
Six Instructions for the Single-Cycle Computer

Operation Code	Symbolic Name	Format	Description	Function	MB	MD	RW	MW	PL	JB	BC
1000010	ADI	Immediate	Add immediate operand	$R[DR] \leftarrow R[SA] + zf\ I(2:0)$	1	0	1	0	0	0	0
0010000	LD	Register	Load memory content into register	$R[DR] \leftarrow M[R[SA]]$	0	1	1	0	0	1	0
0100000	ST	Register	Store register content in memory	$M[R[SA]] \leftarrow R[SB]$	0	1	0	1	0	0	0
0001110	SL	Register	Shift left	$R[DR] \leftarrow sl\ R[SB]$	0	0	1	0	0	1	0
0001011	NOT	Register	Complement register	$R[DR] \leftarrow \overline{R[SA]}$	0	0	1	0	0	0	1
1100000	BRZ	Jump/Branch	If $R[SA] = 0$, branch to $PC + se\ AD$	If $R[SA] = 0$, $PC \leftarrow PC + se\ AD$ If $R[SA] \neq 0, PC \leftarrow PC + 1$	1	0	0	0	1	0	0

Sheet

- Text Book